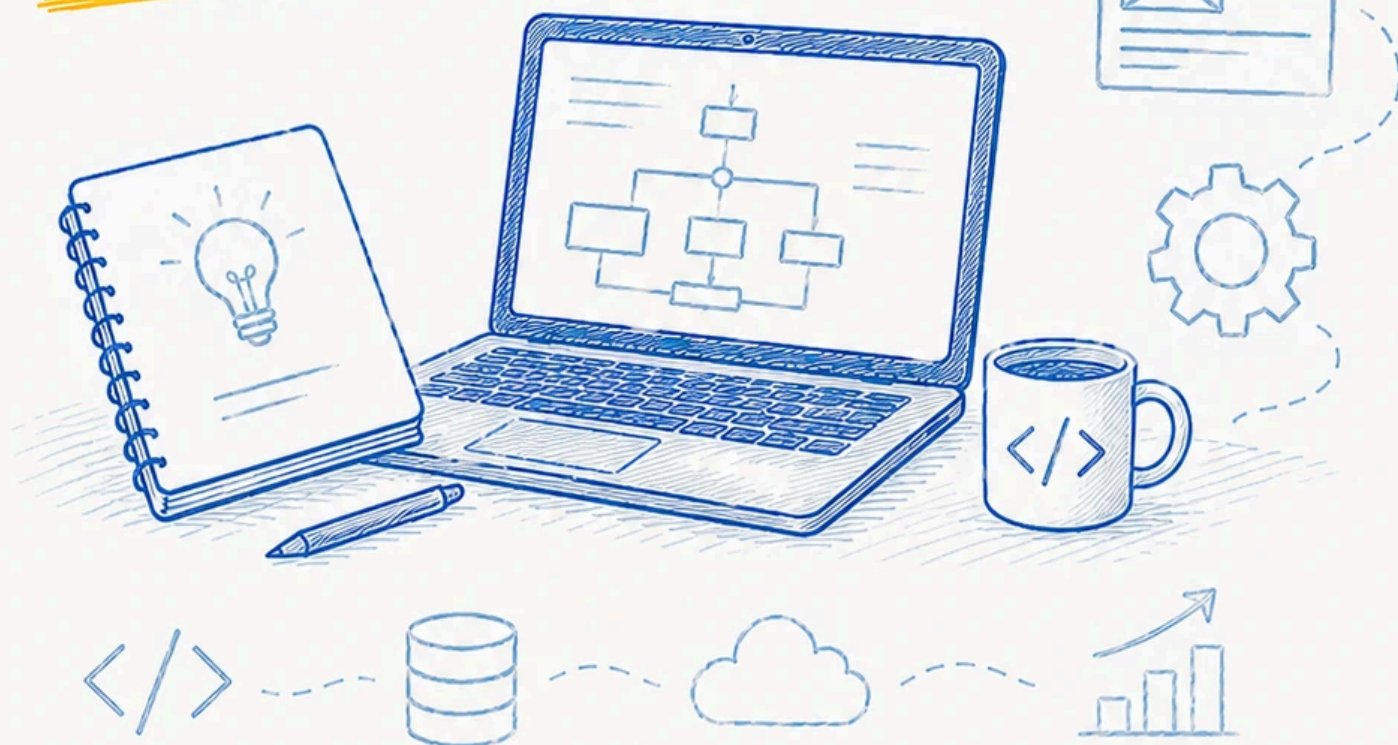


APUNTES

Proyecto intermodular de DAW

Primer y segundo curso



Por

Sergi García Barea

licencia
CC BY SA



Apuntes PI1 — Cómo afrontar el proyecto

Sergi Garcia Barea

CC BY-SA 4.0

Apuntes PI1 — Cómo afrontar el proyecto

Apuntes PI1 — Cómo afrontar el proyecto

 *El día que todo empezó con una idea difusa*

Era septiembre y el profesor soltó la frase: “*Tenéis que hacer un proyecto intermodular*”. Silencio. Miradas de lado. Alguien preguntó: “¿Y podemos hacer lo que queremos?”. “Sí, pero tiene que tener entidad suficiente”.

Ahí estabais: un montón de ideas sin forma, algunas demasiado grandes (una red social mejor que Instagram), otras demasiado pequeñas (una calculadora con cambio de tema). Y un plazo que parecía lejano pero que se echaría encima antes de lo que imaginabais.

Estos apuntes no os van a enseñar a programar mejor. Van a enseñaros a **afrontar el proyecto**: a definir bien qué hacer, a organizaros, a tomar decisiones con criterio, a documentar lo que importa y a llegar a tiempo sin morir en el intento.

¿Qué encontrarás aquí?

PI1 es la primera mitad del proyecto. Empiezas con una idea y terminas con un **MVP funcional** que presentas ante el tribunal. Estas 8 unidades te guían en ese camino:

U1. Definir el proyecto

De la idea difusa a los objetivos concretos. Qué vas a hacer, para quién, y hasta dónde llegas.

→

U2. Diseñar la solución

Dividir el problema en partes. Arquitectura, módulos, responsabilidades.

→

U3. Organizar los datos

Qué información maneja tu proyecto y cómo estructurarla.

→

U4. Documentar y comunicar

Cómo explicar tu proyecto: memorias, informes, presentaciones.

→

U5. Construir la interfaz

Del prototipo en papel a la pantalla que ve el usuario.

→

U6. Implementar la lógica

Cómo priorizar, programar por partes y refactorizar a tiempo.

→

U7. Trabajar en equipo

Git, comunicación, reparto de tareas, y cómo sobrevivir a un grupo.

→

U8. Planificar y organizarse

SCRUM, Kanban, estimaciones, sprints. No es opcional.

→

→

¿Cómo usar estos apuntes?

No están pensados para leerse del tirón. Cada unidad corresponde a un momento del proyecto:

1. **U1** — antes de empezar a codificar
2. **U2-U3** — durante el diseño
3. **U4** — mientras desarrollas (documenta mientras haces)
4. **U5-U6** — durante la implementación
5. **U7-U8** — durante todo el proceso

Ruta de aprendizaje PI1

 No tienes que leerlo todo seguido

Los apuntes están organizados por unidades, pero el proyecto no es lineal. Un día estás diseñando la base de datos y al siguiente descubres que necesitas cambiar algo en la interfaz. Esta ruta te dice qué unidad te ayuda en cada situación, sin tener que memorizar el índice.

Fases del proyecto y unidades recomendadas

Fase 0: Tienes una idea, pero no sabes si vale

U1. Definir el proyecto

- Pasar de “mola hacer X” a “X es viable, concreto y tiene sentido”
- Definir objetivos, alcance, usuarios objetivo
- Decidir si la idea es demasiado grande o demasiado pequeña

Fase 1: Ya sabes qué hacer, pero no cómo

U2. Diseñar la solución + U3. Organizar los datos

- Dividir el proyecto en módulos o componentes
- Decidir qué tecnología usar y por qué
- Modelar los datos que necesitas guardar

Fase 2: Estás programando y te atascas

U5. Construir la interfaz + U6. Implementar la lógica

- Cuándo empezar por el frontend o por el backend
- Cómo priorizar funcionalidades (qué hacer primero)
- Qué hacer cuando una parte se te atasca

Fase 3: Trabajas en grupo y hay conflictos

U7. Trabajar en equipo

- Cómo organizar el repositorio sin pisaros
- Cómo repartir tareas de forma justa
- Qué hacer si alguien no trabaja

Fase 4: Se acerca la fecha y no llegas

U8. Planificar y organizarse

- Cómo estimar cuánto falta
- Cómo priorizar lo que realmente importa
- Técnicas para recuperar el tiempo perdido

Fase 5: Tienes que presentar y no sabes qué poner

U4. Documentar y comunicar

- Qué incluir en la memoria y qué omitir
- Cómo hacer una presentación que no aburra
- Cómo defender tu proyecto ante el tribunal

Para cada situación, una herramienta

Situación	Recurso útil
No sé si mi idea es buena	U1 — Criterios para validar ideas
Me pierdo entre tantas clases y archivos	U2 — Diagramas de arquitectura
No sé qué guardar en la BD	U3 — Modelado conceptual
Odio escribir memorias	U4 — Plantillas y ejemplos
La interfaz queda horrible	U5 — Prototipado rápido
No sé por dónde empezar a programar	U6 — Priorización incremental
Mi compañero no hace nada	U7 — Estrategias de grupo
No voy a llegar a tiempo	U8 — Técnicas de estimación

Ana llegó emocionada a clase. “¡Ya sé lo que voy a hacer! Una red social para mascotas con geolocalización, chat, marketplace, adopciones, y un algoritmo que recomienda parques cercanos según el tipo de perro”.

El profesor asintió, sonrió, y dijo: “Estupendo. ¿Cuánto crees que se tarda en hacer todo eso?”.

Ana calculó: “Dos meses, seguro”.

“Vale. Pues de plazo tienes cuatro meses. Pero además tienes otras 5 asignaturas, exámenes, y una vida más allá de DAW”.

Ese día Ana aprendió que un proyecto no se define por lo que **puede llegar a ser**, sino por lo que **vas a conseguir hacer en el tiempo que tienes**. Y que una red social para mascotas entera no la haría ni Google en un cuatrimestre.

De la idea al proyecto

Toda idea de proyecto empieza siendo difusa. El objetivo de esta unidad es convertir esa chispa inicial en algo **concreto, viable, y planificable**.

El peligro de las ideas difusas

“Una app para gestionar tareas”

¿Gestionar tareas de qué tipo? ¿Para quién? ¿En qué plataforma? ¿Con qué funcionalidades? Una idea así puede derivar en una to-do list de 3 días... o en un JIRA que no terminarías nunca.

Cómo concretar una idea

Para pasar de idea difusa a proyecto definido, responde estas preguntas:

1. **¿Qué problema resuelve?** No es lo mismo “hago una app de recetas” que “ayudo a personas con intolerancias a encontrar recetas adaptadas”.
2. **¿Para quién?** Describe a tu usuario tipo: edad, perfil, necesidades.
3. **¿Qué hace obligatoriamente?** Funcionalidades imprescindibles (las que justifican que tu proyecto existe).
4. **¿Qué haría si tuviera tiempo?** Funcionalidades deseables, pero no críticas.
5. **¿Qué NO va a hacer?** Tan importante como lo que incluyes es lo que excluyes.

El MVP como mentalidad

MVP (Minimum Viable Product / Producto Mínimo Viable) no significa “hacerlo mal”. Significa **hacer solo lo necesario para que el proyecto tenga sentido**. Pregúntate: ¿cuál es la versión más pequeña de mi idea que sigue siendo útil?

► **?** ¿Y si mi MVP se queda en algo demasiado simple?

Objetivos SMART

Los objetivos de tu proyecto deben ser SMART:

Letra	Significado	Ejemplo
S	Específico (Specific)	“Registro con email y contraseña” vs “Sistema de usuarios”
M	Medible (Measurable)	“Soporte para 100 usuarios simultáneos” vs “Que vaya rápido”
A	Alcanzable (Achievable)	Con tus conocimientos actuales o aprendiendo lo justo
R	Relevante (Relevant)	Aporta valor al proyecto, no es relleno
T	Temporal (Time-bound)	“Para la semana 6 del proyecto” vs “Cuando pueda”

Alcance: lo que haces y lo que no

Un error clásico es no definir los límites. Usa esta tabla:

Funcionalidad	Prioridad	Tiempo estimado
Registro e inicio de sesión MVP		1 semana
CRUD de tareas	MVP	2 semanas
Sistema de etiquetas	MVP	3 días
Notificaciones push	Deseable	2 semanas
IA que recomienda tareas	Fuera de alcance	—

► **?** ¿Qué hago si el profesor me dice que mi proyecto es demasiado simple?

Enfoque: abarcar mucho

Tengo mil ideas y voy a implementarlas todas. Total, tiempo hay —o eso creo. Empezaré por lo más difícil y si sobra tiempo, añado el resto.

Enfoque: MVP primero

Identifico las 3-4 funcionalidades clave que definen mi proyecto. Las implemento bien, las pruebo, las documento. Luego, si queda tiempo, añado más.

Caso A — El ambicioso Un equipo decidió hacer una plataforma de cursos online con foros, vídeos, chat en vivo, certificados y recomendaciones personalizadas. A mitad de curso se dieron cuenta de que no tenían ni el login bien hecho. En la presentación mostraron un formulario de registro que fallaba y un reproductor de vídeo que no cargaba.

Resultado: Suspenso. Demasiado alcance, cero ejecución.

Caso B — El enfocado Otro equipo hizo una aplicación para que los vecinos de una comunidad gestionen incidencias (una gotera, una luz fundida). Solo tenía: registro, crear incidencia, listar incidencias, cambiar estado. Todo funcionaba. La interfaz era limpia, el código estaba testeado, la memoria documentada.

Resultado: Notable. Poco alcance, pero todo hecho bien.

La moraleja: **no es lo que prometes, es lo que entregas.**

Errores comunes y cómo evitarlos

Error	Solución
Empezar a codificar sin tener claro el alcance	Dedica una semana solo a definir y planificar
Decir “sí” a cada idea que surge	Mantén una lista de “ideas futuras” fuera del alcance del proyecto
Pensar que “ya irás viendo”	El proyecto sin plan es un barco sin rumbo. Acabarás donde no querías
No consultar con el profesor	Cada 2-3 semanas, valida tu dirección. Mejor corregir a tiempo

U2. Diseñar la solución

 *El día que el código se convirtió en espagueti*

Pablo empezó a codificar el mismo día que le dieron el proyecto. “Total, ya sé lo que hay que hacer —una clase aquí, un método allá—”.

Dos semanas después, tenía 2000 líneas en un solo archivo. Las funciones hacían tres cosas cada una. Cambiar el color de un botón rompía la lógica de búsqueda. “¿Por qué pasa esto?”, se preguntaba.

Porque no diseñó la solución antes de escribir código. Porque pensó que programar era solo escribir, y no es así. Programar es **diseñar** primero para **escribir** después.

Por qué diseñar antes de codificar

Diseñar la solución es como hacer los planos de una casa antes de construirla. Puedes empezar a poner ladrillos directamente, pero probablemente te falten ventanas, los pasillos no lleven a ninguna parte, y el tejado se caiga.

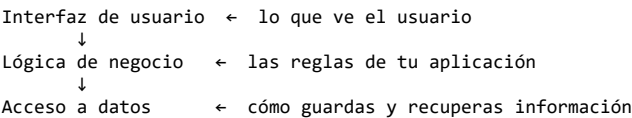
Diseñar tu proyecto significa responder:

- ¿Qué **partes** o **módulos** tiene mi aplicación?
- ¿Cómo se **comunican** entre sí?
- ¿Qué **responsabilidades** tiene cada una?
- ¿Qué **datos** necesita cada parte?

Divide y vencerás

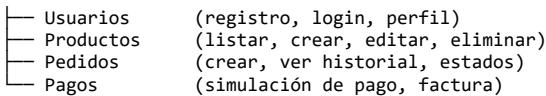
Todo proyecto se puede dividir en partes más pequeñas. La clave está en elegir **cómo** dividirlo.

Por capas (la más común)



Cada capa se comunica solo con la de al lado. Así, si mañana cambias de base de datos, solo tocas la capa de datos. La interfaz sigue igual.

Por funcionalidades



Cada funcionalidad es independiente. Puedes trabajar en una sin romper otra.

►  ¿Y si mi proyecto es pequeño y no necesita tanta estructura?

Cómo documentar el diseño

No necesitas un documento de 50 páginas. Con estos diagramas basta:

Diagrama de arquitectura (cajas y flechas)

Dibuja las partes de tu aplicación y cómo se conectan. Herramientas: **Draw.io** (online, gratis), **Mermaid** (en Markdown), o lápiz y papel.

[Navegador] → [Servidor web] → [API] → [Base de datos]
↓
[Servicio de correo] (para emails)

Diagrama de componentes

Para cada parte, enumera qué hace y qué necesita:

Componente	Responsabilidad	Depende de
Controlador de usuarios	Gestionar registro y login	Servicio de usuarios
Servicio de usuarios	Validar datos, encriptar passwords	Repositorio de usuarios
Repositorio de usuarios	Guardar/recuperar de BD	Base de datos

Diseño mínimo

Empiezo a codificar y voy decidiendo sobre la marcha. Si algo no encaja, lo muevo. Es más rápido al principio.

Diseño planificado

Dedico 2-3 días a dibujar la estructura antes de escribir código. Cuando programo, sé exactamente dónde va cada cosa. Pierdo tiempo al principio, pero lo recupero con creces después.

Decisiones técnicas con criterio

Cuando diseñas tu solución, tendrás que elegir tecnologías. Hazlo con criterio:

1. **No elijas lo que está de moda**, elige lo que resuelve tu problema
2. **No uses una tecnología solo porque la conoces**, a veces merece la pena aprender algo nuevo
3. **No cambies de tecnología a mitad del proyecto** a menos que sea estrictamente necesario

Para cada decisión, pregúntate: - ¿Esta tecnología me ayuda a resolver el problema o me añade complejidad? - ¿Tengo tiempo para aprenderla o voy a estresarme?
- ¿El equipo (si lo hay) la conoce o vamos a perder tiempo?

El equipo que no diseñó nada

Cuatro estudiantes decidieron hacer una app para alquilar apuntes entre alumnos. Se repartieron el trabajo así: - Persona A: pantalla de login - Persona B: pantalla de subir apuntes - Persona C: pantalla de buscar - Persona D: base de datos

Empezaron a codificar el día 1. A las dos semanas, Persona A había hecho el login en React. Persona B había hecho “subir apuntes” en HTML plano. Persona C había hecho la búsqueda con PHP. Persona D había creado una base de datos en MySQL pero sin conectarla a nada.

Cuando intentaron unirlo todo, nada funcionaba. Cada uno había usado tecnologías distintas, estilos diferentes, y los datos no se comunicaban.

Lección: diseñar la solución juntos, **antes** de escribir código, habría evitado meses de frustración. Una tarde dibujando la arquitectura habría ahorrado tres semanas de trabajo inútil.

Qué harían hoy: dedicar 2 días a decidir stack común, definir la estructura de carpetas, acordar cómo se comunican las partes, y hacer un prototipo esqueleto antes de que cada uno trabaje por su lado.

Checklist para tu proyecto

Antes de escribir la primera línea de código:

- ☐ He dividido el proyecto en 3-7 partes o módulos
- ☐ Cada parte tiene una responsabilidad clara
- ☐ He dibujado un diagrama de arquitectura
- ☐ He elegido las tecnologías con criterio
- ☐ El equipo (si existe) está de acuerdo con la estructura
- ☐ He validado el diseño con el profesor

U3. Organizar los datos

El día que los datos se descontrolaron

Laura estaba orgullosa de su aplicación de recetas. Tenía registro, búsqueda, valoraciones... pero algo fallaba. Cuando un usuario se daba de baja, sus recetas favoritas desaparecían. Cuando dos usuarios tenían el mismo nombre, se mezclaban sus datos.

— “Es que no pensé bien cómo guardar la información”, admitió.

Laura había empezado creando una tabla “recetas” y, según necesitaba más cosas, añadía columnas, tablas sueltas, y un par de archivos JSON “temporales” que se volvieron permanentes.

El caos de datos es silencioso: no da error hasta que es demasiado tarde.

Piensa en los datos antes de programar

Antes de escribir una sola función, pregúntate:

- ¿Qué **entidades** existen en mi proyecto? (usuarios, productos, pedidos...)

- ¿Qué **atributos** tiene cada una? (nombre, email, precio, fecha...)
- ¿Cómo se **relacionan** entre sí? (un usuario tiene muchos pedidos)
- ¿Qué **reglas** debo cumplir? (email único, precio positivo...)

De las ideas a las entidades

Coge tu idea de proyecto y extrae los **sustantivos**. Esos son tus candidatos a entidades:

“Una aplicación donde **usuarios** pueden publicar **recetas**, y otros **usuarios** pueden dejar **comentarios** y **valoraciones**”

Entidades: Usuario, Receta, Comentario, Valoracion

Relaciones entre entidades

Conéctalas. Pregunta: “¿cuántos?”:

- Un Usuario tiene muchas Recetas → 1 a N
- Una Receta tiene muchos Comentarios → 1 a N
- Un Usuario deja muchas Valoraciones → 1 a N
- Una Receta tiene muchas Valoraciones → 1 a N

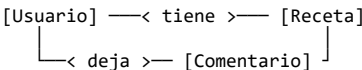
► ? ¿Necesito siempre una base de datos relacional?

Del modelo conceptual al esquema

No necesitas ser experto en bases de datos para hacer un buen diseño. Sigue estos pasos:

Paso 1: Dibuja las entidades y relaciones

Con lápiz y papel o Draw.io:



Paso 2: Define los atributos clave

Para cada entidad, decide qué datos guardar. Empieza por los imprescindibles y añade solo lo que realmente necesitas:

Entidad	Atributos
Usuario	id, nombre, email, password_hash, fecha_registro
Receta	id, titulo, ingredientes, instrucciones, usuario_id, fecha
Comentario	id, texto, usuario_id, receta_id, fecha

Paso 3: Traduce a tablas

Cada entidad → una tabla. Cada atributo → una columna. Las relaciones 1 a N → una clave foránea en la tabla “muchos”.

Enfoque: diseñar primero

Dedico una tarde a dibujar el modelo de datos. Identifico entidades, relaciones y restricciones. Cuando programo, sé exactamente qué tablas crear.

Enfoque: crear sobre la marcha

Empiezo con una tabla y voy añadiendo columnas según las necesito. Si algo no encaja, creo otra tabla. Al final, el esquema es un laberinto.

Buenas prácticas para tus datos

Nombra bien las cosas

- Tablas en plural y snake_case: usuarios, recetas_favoritas
- Columnas descriptivas: fecha_creacion, precio_unitario
- Clave primaria siempre id

No guardes lo que puedas calcular

- Guarda fecha_nacimiento, no edad (la edad se calcula)
- Guarda precio_unitario y cantidad, no precio_total (se calcula)

Protege los datos sensibles

- Contraseñas → hash (bcrypt), nunca texto plano
- Emails → pueden ser identificativos, trátalos con cuidado
- Datos bancarios → no los guardes si no es estrictamente necesario

Una base de datos que creció sin control

Un equipo hizo una aplicación para gestionar eventos. Empezaron con una tabla eventos con columnas básicas. Según avanzaba el proyecto, fueron añadiendo:

- categoria (texto libre → cada uno escribía “música”, “Musica”, “MUSICA”)

- organizador (texto → a veces nombre, a veces email, a veces ID)
- asistentes (texto → lista separada por comas: “1,2,3,4”)
- imagenes (otra tabla que no se llegó a conectar bien)

El resultado: datos inconsistentes, consultas lentas, y bugs difíciles de rastrear. Tuvieron que rehacer la base de datos a mitad de proyecto.

Lección: media hora diseñando el modelo de datos al principio habría ahorrado semanas de trabajo. No hace falta ser experto, solo ordenado.

Preguntas clave para tu proyecto

- ¿Qué pasaría si esta tabla tuviera 10.000 filas?
- ¿Un usuario puede tener 0 elementos de esta entidad?
- ¿Qué reglas debo cumplir (email único, precio mínimo)?
- ¿Qué pasa si elimino un usuario: cascada o solo ese registro?
- ¿Necesito que algún campo sea único o combinación única?

U4. Documentar y comunicar

 *El día que el tribunal no entendió nada*

Marcos había programado durante meses. Su aplicación funcionaba. Pero cuando llegó a la defensa, el tribunal le preguntó:

— “Explicanos la arquitectura de tu solución”.

Marcos sudaba. Había pasado tanto tiempo programando que no sabía explicar lo que había hecho. Señalaba partes del código que ni él recordaba bien. La memoria era un documento que había escrito la noche anterior, con capturas de pantalla mal pegadas y frases copiadas de internet.

Aprobó, pero raspado. Su proyecto era mejor que su presentación. Y eso no debería pasar.

Documentar no es opcional

La documentación no es un trámite aburrido que haces al final. Es la **carta de presentación** de tu proyecto. Una buena memoria puede salvar un proyecto regular. Una mala memoria puede hundir un proyecto excelente.

¿Qué hay que documentar?

En PI1 necesitas:

1. **Memoria del proyecto:** documento formal que explica todo tu trabajo
2. **Presentación oral:** los minutos que tienes para defenderlo
3. **Código comentado:** lo justo para que se entienda

Cómo escribir una buena memoria

Estructura recomendada

No inventes la rueda. Sigue esta estructura:

Sección	Qué poner	Extensión
Introducción	Contexto, problema que resuelve, objetivo	1-2 páginas
Análisis	Requisitos, usuarios, estudio de alternativas	2-3 páginas
Diseño	Arquitectura, diagramas, decisiones técnicas	3-4 páginas
Implementación	Tecnologías, funcionalidades, capturas	3-4 páginas
Pruebas	Qué has probado y cómo	1-2 páginas
Conclusiones	Qué has aprendido, dificultades, trabajo futuro	1 página

Reglas de oro

1. **Capturas de pantalla:** cada funcionalidad importante debe tener una
2. **Diagramas:** un diagrama vale más que mil palabras
3. **Código solo el relevante:** no copies el proyecto entero, solo fragmentos clave
4. **Lenguaje formal pero claro:** ni “luego hice no sé qué” ni “en virtud de lo dispuesto”
5. **Sin faltas de ortografía:** usa correctores. Las faltas restan seriedad

►  ¿Cuánto tiene que ocupar la memoria?

Cómo hacer una presentación que no aburra

La regla del minuto inicial

En el primer minuto de tu presentación, el tribunal decide si le interesa tu proyecto. No empieces con: - “Bueno, nosotros hemos hecho una aplicación sobre...” - “Nuestro proyecto trata de...”

Empieza con: **El problema:** “Cada año, los estudiantes de DAW pierden apuntes...” **La solución:** “Por eso hemos creado una plataforma donde...” **Una demo rápida:** “Enseñémoslo ya”

Estructura de una presentación de 10 minutos

Minuto	Qué hacer
0-1	Enganchar: problema + solución
1-3	Demo rápida (lo más impactante primero)
3-5	Explicar la arquitectura (sin tecnicismos)
5-7	Decisiones técnicas importantes
7-8	Dificultades y cómo las superaste
8-10	Conclusiones y preguntas

Lo que NO hacer

- Leer diapositivas (el tribunal sabe leer)
- Poner 20 líneas de código en una diapositiva
- Usar letra tamaño 10 porque no cabe todo
- Decir “esto se ve mal porque el proyector no funciona”
- Hablar de espaldas al público mirando la pantalla

El proyecto bien hecho que casi suspende por la presentación

Un equipo desarrolló un sistema de gestión para una pequeña biblioteca. El código era limpio, la base de datos estaba bien normalizada, y el testing cubría el 80% del código. Pero:

- La memoria la entregaron con títulos mal formateados y sin índice
- Las capturas eran pequeñas y borrosas
- En la presentación, el portátil no se conectó al proyector y perdieron 5 minutos
- Leían las diapositivas sin mirar al tribunal
- Cuando les preguntaron por una decisión técnica, no supieron explicarla

Resultado: un 5 raspado. El profesor les dijo: “el proyecto merecía más, pero no supisteis vendérmelo”.

Qué harían hoy: ensayar la presentación 3 veces antes, tener un plan B para el proyector, preparar respuestas a preguntas típicas, y dedicar más tiempo a la memoria que al último detalle del código.

Checklist para tu defensa

- ☐ La memoria está completa, maquetada y sin faltas
- ☐ He ensayado la presentación al menos 2 veces
- ☐ Tengo plan B si falla el proyector o el portátil
- ☐ Sé responder por qué elegí cada tecnología
- ☐ Puedo explicar la arquitectura sin mirar apuntes
- ☐ He preparado respuestas a posibles preguntas
- ☐ La demo funciona sin conexión a internet (por si acaso)

U5. Construir la interfaz

El día que la interfaz se interpuso

Carla había programado el backend perfecto: rutas REST, base de datos normalizada, autenticación JWT. Pero la interfaz la había hecho en dos tardes, con colores elegidos al azar y botones que se movían según el navegador.

Cuando se lo enseñó a un compañero, este no sabía ni por dónde empezar. “¿Esto es un botón o un título?”. “¿Dónde hago clic para registrarme?”.

La interfaz es lo primero que ve el tribunal. Si es fea, confusa o rota, da igual lo bien que funcione el backend: la primera impresión ya está perdida.

La interfaz importa (y mucho)

El usuario (y el tribunal) no ve tu base de datos, ni tu clean code, ni tu arquitectura en capas. Ven **pantallas**. Y si las pantallas son confusas, el proyecto parece malo, aunque por dentro sea una maravilla.

Piensa en el usuario, no en ti

Pregúntate para cada pantalla:

- ¿Sabe el usuario qué tiene que hacer aquí?
- ¿Encuentra fácilmente lo que busca?
- ¿Entiende los mensajes (errores, confirmaciones)?
- ¿Puede usar la aplicación sin instrucciones?

►  ¿Y si no se me da bien el diseño visual?

El proceso de construir la interfaz

Paso 1: Prototipo de baja fidelidad

Dibuja tus pantallas en papel o con herramientas como **Balsamiq**, **Figma**, **Excalidraw**. No te preocupes por los colores ni las fuentes. Preocúpate por:

- ¿Qué información muestra cada pantalla?
- ¿Qué acciones puede hacer el usuario?
- ¿Cómo navega de una pantalla a otra?

Paso 2: Prototipo interactivo

Con **Figma** o **Adobe XD** puedes crear maquetas en las que se puede hacer clic. Pásaselas a un compañero y observa si sabe usarlas sin ayuda.

Paso 3: Implementación

Traduce el prototipo a código. Empieza por la estructura HTML, luego los estilos, luego la interactividad con JavaScript.

Mobile-first: diseña primero para pantalla pequeña, luego expande para escritorio. Es más fácil añadir que quitar.

La misma app, dos interfaces

Dos equipos hicieron la misma aplicación: un gestor de tareas.

Equipo A: - Lista de tareas con colores para cada prioridad - Botón “+” visible y grande - Los errores aparecen en rojo al lado del campo - Diseño responsive (se ve bien en móvil y escritorio) - Tiempo invertido en interfaz: 1 semana

Equipo B: - Lista de tareas en texto plano, sin colores - Botón de añadir pequeño y en la esquina - Los errores son alert() de JavaScript - Solo se ve bien en su pantalla (1920x1080) - Tiempo invertido en interfaz: 2 días

Resultado: el Equipo A sacó mejor nota aunque ambos tuvieran la misma funcionalidad. La interfaz cuidada marcó la diferencia.

Lección: la interfaz no es “lo que sobra después de programar”. Es parte del proyecto. Invierte tiempo en ella.

Lo que el tribunal mira en tu interfaz

- **Consistencia:** todos los botones parecen botones, todos los enlaces parecen enlaces
- **Claridad:** sé qué hace cada cosa sin tener que pensar
- **Feedback:** cuando hago clic, pasa algo; si hay error, lo veo
- **Responsive:** se ve bien en móvil, tablet y escritorio
- **Accesibilidad mínima:** contraste suficiente, textos legibles

Checklist para tu interfaz

- ☐ He dibujado las pantallas antes de programarlas
- ☐ Uso un framework CSS para mantener la consistencia
- ☐ Los mensajes de error son claros y están visibles
- ☐ La aplicación funciona bien en móvil (al menos 375px de ancho)
- ☐ He probado la interfaz con al menos una persona que no sea del equipo
- ☐ Los tiempos de carga son aceptables
- ☐ No hay texto cortado, elementos superpuestos ni enlaces rotos

U6. Implementar la lógica

El día que no sabía por dónde empezar

Sergio se sentó frente al ordenador. Tenía el diseño, los datos modelados, las pantallas dibujadas. Pero el editor estaba en blanco.

“¿Empiezo por el registro de usuarios? ¿Por la lista de productos? ¿Primero el frontend o el backend? ¿Y si todo lo que hago hoy lo tengo que cambiar mañana?”

Llevaba una semana sin escribir ni una línea. Paralizado por el miedo a hacerlo mal. Hasta que su profesor le dijo: “empieza por lo más fácil y ve avanzando. El código perfecto no existe, el código que funciona sí”.

El arte de empezar a programar

El mayor obstáculo no es la complejidad técnica, es saber **por dónde empezar**. La clave está en priorizar.

Prioriza por valor, no por dificultad

Haz primero lo que **más valor aporta al proyecto** y que **menos dependencias tiene**. Normalmente, eso significa:

1. **Autenticación básica** — sin login, no hay usuario que use la app
2. **CRUD de la entidad principal** — la operación más básica sobre tu recurso clave
3. **Interfaz mínima** — lo justo para que se vea algo en pantalla
4. **Mejoras iterativas** — refinar lo que ya funciona

Empieza con un esqueleto

No implementes todas las funcionalidades a la vez. Crea una ruta que devuelva un “Hola mundo”. Luego añade autenticación. Luego añade la primera entidad. Ve creciendo por capas.

Iteración 3: Conexión a base de datos real
Iteración 4: CRUD completo de usuarios
Iteración 5: Autenticación JWT
Iteración 6: CRUD de la siguiente entidad
... así hasta terminar

► ? ¿Qué hago si me atasco en una funcionalidad?

Organiza tu código

Estructura de carpetas

No hay una estructura única, pero esta es una base sólida:

```
src/  
├── routes/           # Definición de endpoints  
├── controllers/      # Lógica de cada endpoint  
├── models/           # Definición de datos  
├── middleware/        # Funciones intermedias (auth, validación)  
├── services/          # Lógica de negocio compleja  
├── utils/             # Funciones auxiliares  
└── config/           # Configuración (BD, variables de entorno)
```

Comentarios que ayudan

Comenta el **por qué**, no el **qué**:

```
// ❌ Mal: ya sé que esto suma  
let total = a + b;  
  
// ✅ Bien: explica por qué haces esta operación  
// Sumamos el IVA (21%) al precio base porque la ley lo exige  
let precioFinal = precioBase * 1.21;
```

Refactoriza desde el principio

No esperes a tener todo el código escrito para mejorarlo. Cada vez que veas que una función se repite, extráela. Cada vez que una clase crezca demasiado, divídela.

La refactorización constante evita que el código se convierta en “espagueti”. Y es más fácil refactorizar 100 líneas que 1000.

📄 El estudiante que programó la funcionalidad más difícil primero

Miguel decidió que su proyecto tendría un sistema de recomendación usando machine learning. Pasó tres semanas implementando un algoritmo de recomendación que nunca llegó a funcionar bien. Para entonces, apenas tenía el login y el registro hechos.

En las últimas dos semanas, tuvo que hacer a toda prisa: CRUD de productos, carrito de compra, pasarela de pago simulada, y la interfaz. Todo mal hecho, sin tests, con bugs evidentes.

Resultado: suspendió. Su proyecto prometía mucho, pero entregó poco.

Lección: haz primero lo que sabes que funciona, lo que aporta valor seguro, lo que el tribunal va a ver y probar. Las funcionalidades “avanzadas” son el postre, no el plato principal. Si no tienes un plato principal sólido, el postre no importa.

Qué haría hoy: empezar con el CRUD de productos (su entidad principal), luego el carrito, luego la autenticación, luego la interfaz. Y solo si sobra tiempo, experimentar con el algoritmo de recomendación.

Checklist para tu implementación

- ☐ Tengo una lista priorizada de funcionalidades
- ☐ Empiezo por lo que menos dependencias tiene
- ☐ Cada funcionalidad la termino antes de pasar a la siguiente
- ☐ Refactorizo cuando veo código duplicado
- ☐ Hago commits pequeños y frecuentes (no “cambios varios”)
- ☐ Pruebo cada funcionalidad antes de darla por terminada
- ☐ Si me atasco, salto a otra cosa o pido ayuda

👥 U7. Trabajar en equipo

📖 *El día que git dijo ‘conflict’ y el grupo se rompió*

“Vale, cada uno trabaja en su parte y al final lo juntamos”.

Suena bien, ¿verdad? Es lo que pensaron Marta, Javi y Lucas cuando empezaron su proyecto. Marta haría el frontend, Javi el backend, Lucas la base de datos. Cada uno en su rama de git, sin pisarse.

Dos semanas antes de la entrega, intentaron mergear.

— “Marta, has cambiado el nombre de una columna que yo usaba” — “Javi, tu API devuelve JSON distinto al que esperaba” — “Lucas, has borrado un archivo que los dos necesitábamos”

El merge fue un infierno. Pasaron un fin de semana entero arreglando conflictos. Aprendieron que **trabajar en equipo no es repartirse tareas y no hablar**. Es comunicarse constantemente.

El equipo empieza antes que el código

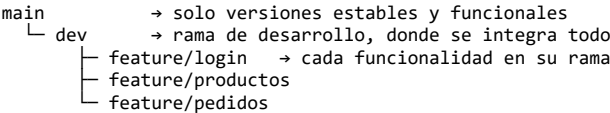
Antes de escribir una línea, el equipo debe acordar:

Reglas básicas

1. **Herramientas de comunicación:** un grupo de WhatsApp o Discord es mínimo. Pero las decisiones importantes no se toman por chat, se hablan.
2. **Reparto de tareas:** quién hace qué. Que sea claro y equilibrado.
3. **Reuniones periódicas:** aunque sean 10 minutos al día para actualizar el estado.
4. **Toma de decisiones:** ¿cómo se decide algo si hay desacuerdo?

Flujo de trabajo con git

El flujo básico para un equipo pequeño:



Reglas: - Nunca commits directamente a main - Cada funcionalidad nueva → rama nueva desde dev - Antes de mergear: code review por otro miembro del equipo
- Merge a dev → probar → merge a main

►  ¿Y si somos dos y no necesitamos tanto flujo?

Código limpio en equipo

Estilo común

Acordad un estilo de código al principio: - ¿Tabulaciones o espacios? - ¿Punto y coma o no? - ¿Comillas simples o dobles? - ¿Nombres en español o inglés?

Usad herramientas como **ESLint** o **Prettier** para que el código de todos tenga el mismo formato automáticamente.

Code review

Revisad el código de los demás antes de mergear. No es para “cazar errores”, es para: - Aprender cómo trabaja tu compañero - Detectar malos entendidos (“ah, yo pensaba que esta función hacía otra cosa”) - Mejorar la calidad entre todos

Commits que se entienden

```
#  Mal
git commit -m "cambios"

#  Bien
git commit -m "Añadir validación de email en formulario de registro"
```

Un buen mensaje de commit explica **qué cambia y por qué**.

El equipo que aprendió a ser equipo

Tres estudiantes empezaron el proyecto con entusiasmo. Cada uno se llevó una parte a casa. Dos semanas después: - Persona A había hecho el login y el registro - Persona B había hecho la gestión de productos - Persona C se había desconectado y no respondía

En la reunión con el profesor, Persona A y B explicaron la situación. El profesor les sugirió: “repartid las tareas de Persona C entre los dos, pero también haced que él se responsabilice de algo concreto y medible”.

Se sentaron, hablaron claro, y rediseñaron el reparto: - Persona A: frontend completo (que ya había mostrado interés) - Persona B: backend completo - Persona C: documentación + tests + despliegue (responsabilidades claras)

A partir de ahí, se reunían 15 minutos cada día (por Discord) para actualizar el progreso. No funcionó perfecto, pero funcionó.

Lección: un equipo no es “cada uno a su bola”. Es comunicación constante, reparto claro de responsabilidades, y saber reaccionar cuando alguien se queda atrás.

Conflictos (de código y de personas)

Conflictos en git

Los conflictos ocurren cuando dos personas modifican la misma parte. No pasa nada. Así se resuelven:

1. git pull la rama destino
2. Busca los marcadores <<<<<<, =====, >>>>>>
3. Decide qué versión conservar (o una combinación)
4. Borra los marcadores
5. Commit

Con práctica, resolver conflictos lleva segundos.

Conflictos entre personas

Si alguien no trabaja, si hay diferencias de criterio, si la comunicación falla:

1. **Habladlo en persona** (no por chat). Cada uno explica su punto de vista.
2. **Buscad un acuerdo** o una solución intermedia.
3. Si no hay acuerdo, **tercera persona** (el profesor, un mediador).

Checklist para tu equipo

- ☐ Hemos acordado herramientas de comunicación
- ☐ Tenemos un repositorio compartido con ramas
- ☐ Cada uno sabe qué tiene que hacer esta semana
- ☐ Nos reunimos al menos 2 veces por semana
- ☐ Hacemos code review antes de mergear
- ☐ Los commits son descriptivos y pequeños
- ☐ Tenemos un plan si alguien se queda atrás

U8. Planificar y organizarse

 *El día que quedaba un mes y solo tenían el login*

Abril. Quedaban 4 semanas para la entrega. El grupo había trabajado, sí, pero sin plan. Cada semana hacían “un poco de todo”: un día frontend, dos días backend, un día cambiaban de idea, otro día no hacían nada.

Cuando miraron atrás, se dieron cuenta de que en 3 meses solo tenían: el login, la base de datos a medio hacer, y tres pantallas sin conectar.

Pánico. Reuniones de emergencia. Algunos fines de semana perdidos.

La falta de planificación no se nota al principio. Se nota al final. Y al final suele ser demasiado tarde.

Por qué planificar (aunque te dé pereza)

Planificar no es hacer documentos bonitos. Es **responder estas preguntas** en cada momento:

- ¿Qué tengo que hacer esta semana?
- ¿Cuánto tiempo me llevará?
- ¿Qué hago si me retraso?
- ¿Estoy avanzando al ritmo correcto?

Sin planificación, avanzas a ciegas. Con planificación, sabes si vas bien, si necesitas acelerar, o si tienes que recortar funcionalidades.

Métodos para organizarte

Kanban (el más visual)

Un tablero con columnas: **Por hacer** → **En proceso** → **Hecho**.

Cada tarea es una tarjeta que se mueve de izquierda a derecha.

Por hacer	En proceso	Hecho
CRUD usuarios	Login (Javi)	Diseño BD
Pantalla inicio	Perfil (Marta)	Prototipo
Tests	—	—

Herramientas: Trello, GitHub Projects, Notion, un pizarra física.

SCRUM adaptado a 2-3 personas

No necesitas un Scrum Master ni sprints de 2 semanas perfectos. Pero sí:

1. **Sprint:** bloque de 1-2 semanas con objetivos concretos
2. **Reunión de planificación:** qué vamos a hacer este sprint
3. **Reunión diaria** (15 min): qué hice ayer, qué haré hoy, bloqueos
4. **Revisión:** al final del sprint, qué se consiguió

►  ¿No es demasiado para un proyecto de clase?

Cómo estimar bien

Estimar es difícil porque siempre tendemos al optimismo. “Esto lo hago en 2 días” → en realidad son 5.

Técnica: desglose y multiplica

1. Divide la tarea en subtareas de máximo 4 horas
2. Estima cada subtask
3. Multiplica el total por **2** (para cubrir imprevistos)

Ejemplo: “Hacer pantalla de registro”

Subtarea	Estimación
Maquetar HTML	2h
Estilos CSS	1h
Validación JS	2h
Conectar con API	2h
Probar	1h
Total	8h → 16h (2 días)

El multiplicador por 2 no es pesimismo. Es experiencia. Los imprevistos, las dudas, los “y si...”, las interrupciones ocurren siempre.

Sin planificar

Voy haciendo tareas según me apetece o según surgen. Al final del mes, no sé si estoy a tiempo. Me estreso y termino haciendo mal las cosas.

Con planificación ligera

Cada semana sé lo que tengo que hacer. Si me retraso, lo veo a tiempo y puedo ajustar. Al final del proyecto, sé que entregaré lo planeado.

Cómo recuperar el tiempo perdido

Si te das cuenta de que no llegas (y te darás cuenta si planificas):

1. **Prioriza:** haz solo lo que realmente importa para el MVP
2. **Recorta:** lo que no sea imprescindible, fuera
3. **Simplifica:** una funcionalidad simple que funciona > una compleja a medias
4. **Pide ayuda:** al profesor o a compañeros
5. **No te engañes:** si ves que no llegas, actúa, no esperes

El proyecto que se salvó por un tablero Kanban

Un equipo de tres empezó usando solo un grupo de WhatsApp para organizarse. “Hoy hago esto”, “mañana haré lo otro”. Sin prioridades claras, sin fechas.

A mitad de curso, un profesor les recomendó usar **GitHub Projects**. Se tomaron una tarde para: - Listar todas las tareas pendientes - Priorizarlas (imprescindible / deseable / futuro) - Asignar responsables - Poner fechas límite

Lo que descubrieron les sorprendió: tenían el doble de tareas de las que pensaban y la mitad del tiempo para hacerlas. Recortaron funcionalidades, reordenaron prioridades, y se centraron en lo importante.

Resultado: entregaron a tiempo, con un proyecto completo y funcional. No sobraron lujos, pero no faltó lo esencial.

Lección: ver el trabajo reflejado en un tablero no estresa. Libera. Porque lo que no está escrito no existe, y lo que no está planificado no se hace.

Checklist para tu planificación

- ☐ Tengo un tablero con todas las tareas del proyecto
- ☐ Cada tarea tiene responsable y fecha estimada
- ☐ Tengo las funcionalidades priorizadas (MVP vs deseable)
- ☐ Sé lo que tengo que hacer esta semana
- ☐ Hago reuniones de equipo al menos cada 3-4 días
- ☐ Si veo que me retraso, ajusto el plan inmediatamente
- ☐ Cada sprint entrego algo funcional, no “código a medias”